Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=dsUB4bst9S.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Lemao Liu, Masao Utiyama, Andrew Finch, and Eiichiro Sumita. Agreement on target-bidirectional neural machine translation. In Kevin Knight, Ani Nenkova, and Owen Rambow (eds.), *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 411–416, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1046. URL https://aclanthology.org/N16-1046.

Sanidhya Mangal, Poorva Joshi, and Rahul Modak. Lstm vs. gru vs. bidirectional rnn for script generation. *arXiv preprint arXiv:1908.04332*, 2019.

Ida Momennejad, Hosein Hasanbeig, Felipe Vieira Frujeri, Hiteshi Sharma, Nebojsa Jojic, Hamid Palangi, Robert Ness, and Jonathan Larson. Evaluating cognitive maps and planning in large language models with cogeval. *Advances in Neural Information Processing Systems*, 36, 2024.

Lili Mou, Rui Yan, Ge Li, Lu Zhang, and Zhi Jin. Backbone language modeling for constrained natural language generation. *CoRR*, abs/1512.06612, 2015. URL http://arxiv.org/abs/1512.06612.

Anh Nguyen, Nikos Karampatziakis, and Weizhu Chen. Meet in the middle: A new pre-training paradigm. *Advances in Neural Information Processing Systems*, 36, 2024.

Vassilis Papadopoulos, Jérémie Wenger, and Clément Hongler. Arrows of time for large language models. *arXiv preprint arXiv:2401.17505*, 2024.

ME Peters, M Neumann, M Iyyer, M Gardner, C Clark, K Lee, and L Zettlemoyer. Deep contextualized word representations. corr. *arXiv preprint arXiv:1802.05365*, 2018.

Clayton Sanford, Daniel Hsu, and Matus Telgarsky. Transformers, parallel computation, and logarithmic depth. *arXiv preprint arXiv:2402.09268*, 2024.

Dmitriy Serdyuk, Nan Rosemary Ke, Alessandro Sordoni, Adam Trischler, Chris Pal, and Yoshua Bengio. Twin networks: Matching the future for sequence generation, 2018. URL https://arxiv.org/abs/1708.06742.

Adam S. Shai, Sarah E. Marzen, Lucas Teixeira, Alexander Gietelink Oldenziel, and Paul M. Riechers. Transformers represent belief state geometry in their residual stream, 2024. URL https://arxiv.org/abs/2405.15943.

Shai Shalev-Shwartz, Ohad Shamir, and Shaked Shammah. Failures of gradient-based deep learning. In *International Conference on Machine Learning*, pp. 3067–3075. PMLR, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Sean Welleck, Kianté Brantley, Hal Daumé Iii, and Kyunghyun Cho. Non-monotonic sequential text generation. In *International Conference on Machine Learning*, pp. 6716–6726. PMLR, 2019.

Zhirui Zhang, Shuangzhi Wu, Shujie Liu, Mu Li, Ming Zhou, and Tong Xu. Regularizing neural machine translation by target-bidirectional agreement, 2018. URL https://arxiv.org/abs/1808.04064.

A Statement Of Contributions

Edward Hu developed the BST, set up the code and infrastructure, proposed and ran the experiments. Kwangjun Ahn developed the BST, assisted with the Stargraph experiments, and contributed to theoretical analysis. Qinghua Liu contributed the parity proof and gave feedback to the project. Haoran Xu assisted in debugging, ran baselines, and gave feedback. Manan Tomar contributed early architecture code, ran the probe analysis, and gave feedback. Ada Langford contributed code for the Stargraph experiment. Dinesh Jayaraman advised EH and gave feedback. Alex Lamb oversaw the project, developed the BST, debugged models, proposed experiments, and ran the probe analysis. John Langford oversaw the project, developed the BST, contributed to BST theory, proposed experiments, and assisted with writing.

B Proofs of Theoretical Results

B.1 Proof of Theorem 1

In this section, we provide a formal statement and a proof of Theorem 1.

As discussed in the main text, the starting point of our argument is the observation by (Bachmann & Nagarajan, 2024, Appendix F.2) that the model quickly learns the aforementioned "flawed cheat" solution, which prevents it from learning the true solution. Specifically, next-token predictors rapidly adopt this flawed shortcut, but make little progress in correctly predicting the first vertex on the path. Below, we provide formal evidence that learning the flawed cheat indeed hinders the model from learning the true solution.

In essence, once the flawed cheat solution is perfected, the model receives supervision only from the prediction of the first vertex after the starting node. This effectively reduces the star graph task to the following simplified task:

Task 1. *Given a stargraph and a goal node, predict the correct neighbor of the starting node.*

The formal statement of Theorem 1 is that Task 1 is at least as difficult as learning the full parity function (i.e., predicting whether the sum of the elements in a binary string is even or odd).

Theorem 5. *For every full parity problem, there exists a stargraph such that solving Task 1 provides a solution to the parity problem.*

Notably, empirical evidence shows that gradient-based methods struggle to learn the full parity function in standard training setups, especially in high dimensions (e.g., Shalev-Shwartz et al., 2017; Abbe & Sandon, 2023). It has been also conjectured that learning the full parity function requires a number of samples and computations exponential in the input dimension (e.g., Abbe & Boix-Adsera, 2022). Consequently, Task 1 inherits this difficulty, as it encapsulates learning the full parity function as a special case.

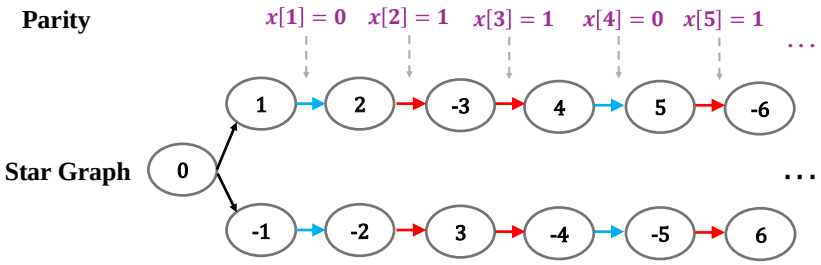


Figure 9: Reduction from learning parity to learning star graph. Note that if parity$(x) = 0$ (even), then the shortest path from center 0 to vertex n passes 1. Otherwise, it passes -1. As a result, solving the star graph instance induced by x implies correctly predicting the parity of x.

Proof of Theorem 5. Consider a star graph with $2n+1$ vertices. For simplicity, we index the vertices by $-n, \ldots, -1, 0, 1, \ldots, n$. Given a binary string $x \in \{0,1\}^n$, we generate a star graph instance as

follows: First we make vertex 0 the center and connect it to vertex ± 1. For each $i \in \{1, \ldots, n-1\}$, if $x[i] = 0$ we connect vertex i to $i+1$ and vertex $-i$ to $-i-1$. Otherwise, we connect i to $-i-1$ and $-i$ to $i+1$. We illustrate this reduction in Figure 9.

The parity of x can now be determined by the first step towards target vertex n. Specifically, if $\text{parity}(x) = 0$ (even), then the first vertex is 1. Otherwise, it is -1. Consequently, if an algorithm can solve Task 1 for star graphs generated in this way, then it equivalently solves learning the full parity problem. □

B.2 Proof of Theorem 3

We prove this theorem by constructing a counterexample. Consider the distribution D that is uniform over the sequences $\{ACA, BCB\}$. Our goal is to show that there exists an ideal next-token predictor, in the sense of Equation (5), that does not learn a belief state.

First, note that an ideal next-token predictor achieves a log loss of 1 bit on the first token and 0 log loss on all subsequent tokens. Define the forward encoder $\mathsf{F}(x_{1:t})$ to output a two-dimensional vector for any partial sequence $x_{1:t}$. Specifically, we define the encoding as follows:

$$\begin{aligned} \mathsf{F}(\emptyset) &= (-1, -1), \\ \mathsf{F}(A) = \mathsf{F}(B) &= (-1, 1), \\ \mathsf{F}(AC) &= (1, -1), \\ \mathsf{F}(BC) &= (1, 1). \end{aligned}$$

Next, we define the output head T for the next-token predictions:

$$\begin{aligned} \mathsf{T}(-1, -1) &= \text{uniform}(A, B), \\ \mathsf{T}(-1, 1) &= C, \\ \mathsf{T}(1, -1) &= A, \\ \mathsf{T}(1, 1) &= B. \end{aligned}$$

It can be verified that this setup achieves the optimal performance: a 1-bit log loss on the first token (since A and B are equally likely), and 0 log loss on subsequent tokens (since the continuation of the sequence is fully deterministic). However, the key observation is that $\mathsf{F}(A) = \mathsf{F}(B) = (-1, 1)$ is not a belief state for the remainder of the sequence. This is because any function $g(-1, 1)$ applied to this encoding outputs a distribution that is independent of whether the initial token was A or B. Thus, the encoding fails to capture the information necessary to distinguish between the two possible continuations of the sequence, violating the definition of a belief state. Therefore, this next-token predictor does not output a belief state, completing the proof.

B.3 Proof of Theorem 4

We again use a counterexample construction to prove this theorem. Consider the distribution D, which is uniform over the four sequences $\{DAA, DBB, SAB, SBA\}$. Here, the initial tokens D and S determine whether the next symbol is doubled or not. Our goal is to demonstrate that a multi-token model, in the sense of Equation (6), does not learn a belief state.

First, we construct an ideal multi-token model that suffers a log loss of 2 bits on each token except for the last token. Note that this is the minimal log loss achievable. Define the forward encoder $\mathsf{F}(x_{1:t})$ to output a two-dimensional vector for any partial sequence $x_{1:t}$. Specifically, we define the encoding as follows:

$$\begin{aligned} \mathsf{F}(\emptyset) &= (-1, -1), \\ \mathsf{F}(D) = \mathsf{F}(S) &= (-1, 1), \\ \mathsf{F}(DA) = \mathsf{F}(SB) &= (1, -1), \\ \mathsf{F}(DB) = \mathsf{F}(SA) &= (1, 1). \end{aligned}$$

Now, define the output head for predicting the next tokens at different time steps as follows:

$$\begin{aligned}
\mathsf{T}_1((-1,-1)) &= \text{uniform}(S, D),\\
\mathsf{T}_2((-1,-1)) &= \text{uniform}(A, B),\\
\mathsf{T}_3((-1,-1)) &= \text{uniform}(A, B),\\
\mathsf{T}_1((-1,1)) &= \text{uniform}(A, B),\\
\mathsf{T}_2((-1,1)) &= \text{uniform}(A, B),\\
\mathsf{T}_1((1,-1)) &= A,\\
\mathsf{T}_1((1,1)) &= B.
\end{aligned}$$

This multi-token model clearly achieves the minimal log loss. However, the key observation is that $\mathsf{F}(S) = \mathsf{F}(D) = (-1, 1)$ is not a belief state for the remainder of the sequence. The reason is that any function $g(-1, 1)$ applied to this encoding outputs a distribution that is independent of whether the initial token was S or D. Thus, the encoding fails to capture the necessary information for predicting the future tokens. Therefore, this multi-token model does not output a belief state, completing the proof.

C Star Graph Details

In this section, we provide detailed descriptions of the star graph experiments from Section 3.4.

- **Data Generation.** We use the data generation code from the official codebase. To generate the star graph structure, each node n_i is sampled uniformly from the set $\{1, \ldots, N\}$. For all experiments, we set $N = 50$, and we generate 8M examples for each data set.
- **Data Tokenization.** We follow the same tokenization settings as in (Bachmann & Nagarajan, 2024, Section G.1).
- **Architectures.** Both the forward and backward encoders consist of $n_{\text{layers}} = 6$ layers with an embedding dimension of 768, $n_{\text{head}} = 8$ attention heads, and an MLP expansion factor of 1. The baselines use the same configuration.
- **Training Details.** In all cases, we use the AdamW optimizer with a weight decay strength of 0.1. For $G(2, 5)$, the learning rate is set to $\eta = 3 \cdot 10^{-4}$, while for $G(5, 5)$ and $G(2, 20)$, a smaller learning rate of $\eta = 1 \cdot 10^{-4}$ is used. We run all experiments for 100 epochs to ensure convergence. All models run quickly, finishing 100 epochs in less than 2 hours. The belief state transformer reaches $\approx 100\%$ accuracy in a few minutes on the easier graphs. Each model is trained on a single A100 / H100 GPU with 80GB memory.
- **Evaluation.** We evaluate the models on 10,000 unseen graphs. To succeed, the model must output the entire path correctly, and we report the success rate.

C.1 Baseline details

We use the official codebase from Bachmann & Nagarajan (2024) to instantiate and train the baselines.

- **Forward-only.** This baseline trains a transformer to do next-token prediction over the sequence data. Teacher forcing is used during training, so the model gets ground truth intermediate sequences as input.
- **Data Augmentation.** Here, our data augmentation process entails simply modifying the data using some domain knowledge, before feeding it into the standard forward-only transformer training process.
  Given the original training sequences $[\mathcal{E} \mid n_1, n_l \mid n_1, n_2, n_3, \ldots n_l]$, we propose to replace the task specification n_1, n_l with n_1, n' where $n' \sim \{n_2 \ldots n_{l-1}\}$ is an intermediate node in the path. The resulting training sequence would then look like: $[\mathcal{E} \mid n_1, n' \mid n_1, n_2, n_3, \ldots n_l]$. The idea here is that replacing the long horizon goal with a subgoal closer to the start would make learning the goal conditioning easier.
- **Fill-in-the-Middle.** This is similar to the domain augmentation baseline, except we replace intermediate nodes with intermediate suffixes $n_j, \ldots n_T$. The input to the transformer is then: $[\mathcal{E} \mid n_1, n_T \mid n_j, n_{j+1}, \ldots n_l \mid n_1, n_2, n_3, \ldots n_l]$. During training time, we uniformly sample suffixes of different lengths.